



INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY

H Y D E R A B A D

AIML PG-Level Capstone Project

Real-World RAG (Retrieval-Augmented Generation)

Building, Testing and Tuning Domain-Specific RAG Pipelines

Batch: 26 : Sridhar Vedula, Prerna Saratkar, Rinku Rajput & Veenu Dandekar

Mentors: Gopichand, Lokesh

Project Supervisor: Dr. Manish Shrivastava

Repository: [veenulearns-lab/RAGBench-Capstone-Batch26](https://github.com/veenulearns-lab/RAGBench-Capstone-Batch26)

Executive Summary

Retrieval-Augmented Generation (RAG) is widely used to improve the factual accuracy of Large Language Models by combining document retrieval with answer generation. While modern RAG systems contain several configurable components, there is limited evidence regarding which components contribute most to performance improvements across different domains.

This project investigates the impact of five major RAG pipeline components: chunking, embedding models, retrieval strategy, context ordering, and answer generation. Using the RAGBench benchmark and RGB robustness benchmark, we evaluated domain-specific RAG pipelines across all five RAGBench domains — Customer Support, Legal, Finance, General Knowledge and Biomedical. Every domain ran the same 96-configuration grid and was scored under the same protocol, with llama-3.3-70b-versatile generating and openai/gpt-oss-120b judging.

A total of 96 configurations were evaluated per domain through systematic experimentation. Performance was measured using the TRACe framework, consisting of Relevance, Utilization, Completeness, and Adherence metrics, together with robustness testing using RGB benchmarks.

The study produced several key findings:

- Chunking was the most influential component in four of the five evaluated domains.
- Context ordering was the strongest contributor in Finance.
- Retrieval strategy and reranking had less impact than initially expected.
- Domain-specific embedding models were beneficial only in selected domains, particularly Legal and Finance.
- Evaluation methodology is itself a critical factor; the judge model and judge prompt affected reported scores more than many architectural design choices.
- Small-sample evaluations significantly overestimated final performance, reinforcing the importance of large-scale validation.

The final outcome of the project is a set of domain-specific RAG configurations, a deployable Gradio-based application, a validated evaluation pipeline, and a collection of methodological lessons regarding trustworthy RAG assessment.

Beyond improving answer quality, this work highlights the importance of validating evaluation instruments before optimizing system architecture. The project demonstrates that reliable measurement is as important as model selection when building production-ready RAG systems.

Table of Contents

1. Problem Statement	5
1.1 What We Actually Set Out to Study	5
2. Literature Review	6
2.1 RAGBench — Where Our Data and Our Metrics Come From	6
2.2 RGB — Testing What Happens When Retrieval Fails	7
2.3 EMNLP 2024 — Best Practices in RAG	7
2.4 Comparison of the Three Sources	7
2.5 The Gap We Are Filling	8
3. Dataset Description	8
3.1 RAGBench	8
3.2 RGB	9
3.3 How We Sampled	9
4. Methodology	9
4.1 System Architecture	10
4.2 Technical Design Decisions	11
4.3 Pipeline Components	12
4.4 How the Experiments Were Run	13
4.5 Tools and Frameworks	15
5. Evaluation Metrics	15
5.1 The Four TRACe Metrics	15
5.2 Rules We Follow When Judging	16
5.3 Judge Validation Rationale	17
5.4 The Judge Must Be Stronger Than the Generator	17
5.5 Reporting	18
6. Deployment	18
6.1 Deployment Stack	18
6.2 Deployment Flow	19
7. RAGBench: Implementation and Results	19
7.1 Which Component Actually Mattered	19
7.2 Customer Support (TechQA)	20
7.3 Legal (CUAD)	21

7.4 Finance (FinQA)	21
7.5 General Knowledge (HotpotQA)	22
7.6 Biomedical (CovidQA)	22
7.7 Summary of Locked Configurations	23
7.8 Why 25-Example Results Cannot Be Reported	24
8. Challenges, Mistakes and What We Learned	25
8.1 The Judge Prompt Mattered More Than Any Design Choice	25
8.2 Every Bug We Found and Fixed	25
8.3 Practical Problems and How We Worked Around Them	26
8.4 Why Some AUCROC Scores Sit Close to Chance	27
9. RGB: Robustness Evaluation	27
9.1 What RGB Tests, and Why It Matters	28
9.2 Dataset	28
9.3 Methodology.....	28
9.4 Results.....	29
9.5 What the RGB Results Show.....	29
9.6 Challenges in the RGB Work	30
10. Impact and Industry Relevance	30
11. Future Scope	31
11.1 Finishing What Is Started	31
11.2 Making the Numbers More Trustworthy.....	31
11.3 Testing the Label Explanation	31
11.4 Improving the Instruction Rather Than the Model	31
11.5 Longer-Term Directions	32
12. Threats to Validity.....	32
12. Conclusion	33
13. References	33
Appendix A — Settings Held Constant.....	35
Appendix B — Environment and Secrets	35
Appendix C — Reproducing a Result.....	36

1. Problem Statement

A large language model only knows what it saw while it was being trained. That creates three problems when you try to use one for real work.

- **It makes things up.** The model has no evidence for the question, but it answers anyway. The answer reads well and sounds confident, and parts of it are simply invented.
- **Its knowledge goes stale.** Anything that changed after the training date is either missing or wrong, and the model has no way of knowing which.
- **It cannot see your documents.** It cannot answer from your contracts, your support articles or your filings, because it never saw them.

In everyday chat these are annoyances. In customer support, medical questions, legal advice or financial summaries they are serious, because a confident wrong answer is worse than no answer at all.

Retrieval-Augmented Generation (RAG) is the standard fix. Instead of relying on what the model memorised, you search your own documents when the question is asked, hand the model the passages you found, and tell it to answer only from those. The model stops trying to remember and starts reading.

1.1 What We Actually Set Out to Study

Building a working RAG pipeline is no longer hard. There are libraries for every stage, and a first version can be running in an afternoon. The hard part, which we ran into almost immediately, is knowing which part of the pipeline is responsible when the answers get better or worse.

A RAG pipeline has at least five moving parts, and each one has several reasonable options:

Stage	Symbol	What it decides	Options we tested
Chunking	K	How documents are cut into searchable pieces	4 per domain, from whole document to word windows
Embedding	ϵ	How text is turned into numbers for searching	2 per domain, one general and one specialist
Retrieval	ρ	How the pieces are found	Dense (meaning-based) or hybrid (meaning + keyword)
Reranking and ordering	ω	Which pieces survive and where they sit in the prompt	Rerank on/off × three orderings
Generation	γ	Which model writes the answer	llama-3.3-70b-versatile

Table 1 — The five stages of a RAG pipeline. A full configuration is the combination of one choice from each row.

Change several of these at once, see the score move, and you have learned nothing about which change did it. That is what this project is really about. We wanted three things:

1. **Attribution.** Which stage is actually holding the pipeline back? Not which stage we assume matters, but which one carries the variation when we measure it.
2. **Generality.** Is the best configuration the same for every domain? If it is, one recommended pipeline is enough. If it is not, then every study that tunes on one dataset and publishes a general recommendation is overstating its case, and the per-domain configuration becomes the real deliverable.
3. **Validity.** Are our own measurements trustworthy? An answer to the first two questions is worthless — worse than worthless, because it looks usable — if the measuring instrument is broken.

We came into this from non-technical backgrounds and learned the material by building it. That shaped the project in a way we think is worth stating: we assumed less than an experienced team might have, checked more, and several of our most useful findings came from noticing that a number looked impossible rather than from designing a clever experiment.

2. Literature Review

Three pieces of published work shaped how we built and measured this system. We summarise each below, in our own words, and note what we took from it.

2.1 RAGBench — Where Our Data and Our Metrics Come From

RAGBench (Friel, Belyi and Sanyal, 2024) is a benchmark of roughly 100,000 question-answer examples drawn from five working domains: biomedical, legal, finance, customer support and general knowledge. Each example gives you a question, the documents that were retrieved for it, an answer, and — this is the part that made it valuable to us — a set of reference scores under a framework called TRACe.

TRACe breaks answer quality into four separate questions rather than one overall score. Was the retrieved material actually about the question? Did the answer use it? Did the answer use all of the relevant material? And is every claim in the answer supported by something in the documents? Because the reference values are supplied, we could check our own scoring against theirs instead of just reporting numbers and hoping.

Two details of how RAGBench was built ended up mattering a lot to us. First, the reference scores were produced by GPT-4 using a specific prompt, printed in full in the paper's Appendix 7.4. Second, the answers being scored were written by deliberately weaker models at a high

temperature, using a loose prompt that did not insist on sticking to the documents. That was on purpose — a hallucination benchmark needs hallucinations in it. It also means the benchmark was designed with a strong judge grading weaker writers, which is exactly the arrangement we eventually adopted.

2.2 RGB — Testing What Happens When Retrieval Fails

RGB (Chen et al., 2024) asks a different question. TRACe measures how good an answer is when the retrieval worked. RGB measures how the model behaves when retrieval goes wrong, which in production is most of the time. It tests four specific abilities: coping with irrelevant documents mixed into the results, refusing to answer when the evidence genuinely is not there, combining evidence spread across several documents, and spotting factual errors planted in retrieved text. We treated RGB as the necessary second half of the evaluation and have given it its own chapter.

2.3 EMNLP 2024 — Best Practices in RAG

Wang et al. (2024) tested many combinations of RAG components and reported which combinations worked. Their headline conclusions were that hybrid retrieval with reranking performs well in general, that the right component choices differ by domain, and that there is a real trade-off between quality and speed. The middle claim is the one we set out to test properly, because it is asserted often and demonstrated rarely.

2.4 Comparison of the Three Sources

	RAGBench	RGB	EMNLP 2024 Best Practices
Main purpose	Measure end-to-end answer quality on realistic data	Stress-test specific model abilities under difficult conditions	Find which component combinations work well
Domains	Biomedical, legal, finance, customer support, general knowledge	General news (English and Chinese)	Domain-agnostic, tested on several corpora
Size	~100,000 annotated examples	~1,000 examples across four ability tests	~300 pipeline configurations tested
Metrics	TRACe: relevance, utilization, completeness, adherence	Accuracy, rejection rate, error detection and correction	Factuality, relevance, efficiency

Strength	Reference labels let us validate our own scoring	Isolates one failure mode at a time	Practical, component-level guidance
Limitation	Labels describe the benchmark's own answers, not ours (see §8.4)	Constructed questions, not real production traffic	No single winning recipe; needs local tuning
What we took	Our datasets, our metrics, and our judge prompt	Our robustness evaluation (Chapter 9)	The domain-specificity claim, which we tested

Table 2 — How the three sources relate to each other and to our work.

2.5 The Gap We Are Filling

Reading these together, four things stood out. Published comparisons usually change one component while leaving the others at unstated defaults, so their effect sizes cannot be compared across papers. The generator is often held fixed and treated as background, even though it is one of the biggest levers. Results are established on one dataset and then described as general. And the validity of LLM-produced scores is rarely questioned at all — which, as Chapter 8 describes, turned out to be the single largest source of error in our own work.

3. Dataset Description

3.1 RAGBench

RAGBench is published on HuggingFace as `rungalileo/ragbench`. We used the test split of four component datasets, one per domain. Each example arrives with its own documents already attached, which is different from a normal production system where one big index serves every question. That detail shaped our whole implementation, because it means the index has to be rebuilt for every single question.

Domain	Dataset	What the documents look like	What makes it hard
Customer Support	techqa	Short technical support notes	Articles overlap heavily — telling them apart is harder than finding them
Legal	cuad	Commercial contracts	Documents run 8,000–109,000 characters, with numbered clauses
Finance	finqa	Financial reports with tables and figures	A fact is a number plus the sentence that qualifies it; split them and the fact is destroyed

General Knowledge	hotpotqa	Wikipedia passages	Questions need two or more documents combined, and distractors are planted deliberately
Biomedical	Covidqa	COVID-19 research papers, abstracts and scientific literature	Technical medical terminology, long evidence passages, and a high risk of hallucinations require accurate retrieval and evidence grounding.

Table 3 — The four RAGBench subsets used in the final sweep, and the specific difficulty each one presents.

A note on coverage. RAGBench has five domains and all five are reported in Chapter 7. Each one ran the identical 96-configuration grid under the identical judge protocol, so the domains are directly comparable — nothing differs between them except the documents and the two candidate embedding models each domain was given.

3.2 RGB

RGB supplies 300 English questions per ability test, with a matching Chinese set that we did not use. It ships three data files: the base set, an information-integration set, and a counterfactual set in which retrieved documents have been deliberately corrupted. Noise is introduced by mixing a controlled proportion of irrelevant passages into the retrieved documents, which lets you sweep difficulty from clean to almost entirely noise.

3.3 How We Sampled

Every evaluation subset is drawn with a fixed random seed of 42. A seed is the starting point for a computer's random number generator; fix it, and you get the same "random" selection every time. This means every experiment inside a domain saw exactly the same questions, so a difference in score comes from the pipeline change and not from one run happening to draw easier questions. We ran three scales: a single example to check the code runs, 25 examples to screen configurations, and 200 examples for the final result. Only the 200-example numbers are reported as results, for reasons explained in §7.5.

4. Methodology

Our aim was to build one pipeline in which every stage could be swapped by changing a setting rather than by editing code, run the same grid of settings across every domain, and score the results in a way we could defend. The design was also shaped by three practical constraints we could not

avoid: no GPU, hosted models with daily usage limits, and Colab sessions that disconnect without warning.

4.1 System Architecture

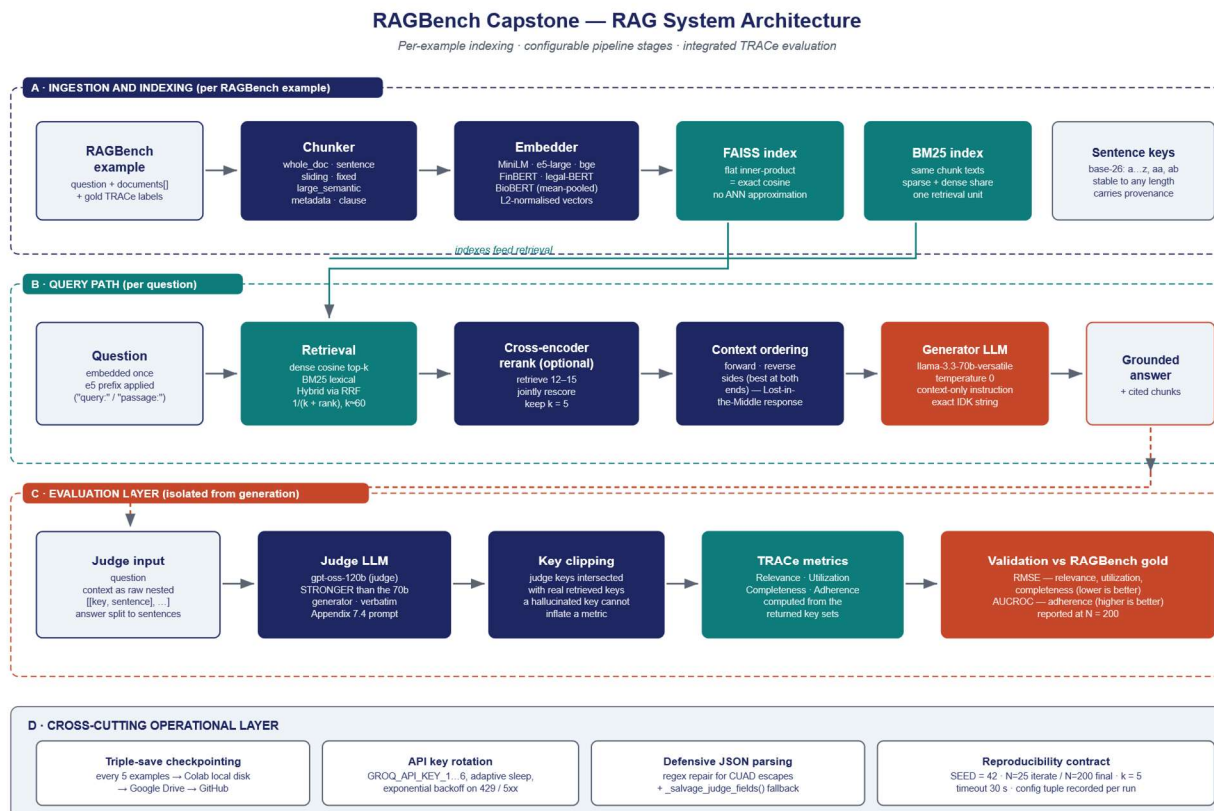


Figure 1 — The full system. Band A prepares and indexes documents, Band B answers a question, Band C scores the answer, and Band D is the operational scaffolding that keeps long runs alive.

Reading the Diagram

Band A — preparing and indexing. Because every RAGBench example brings its own documents, we build the index fresh for each question rather than once for a whole corpus. The documents are cut into chunks by whichever of the four strategies the configuration names, and each chunk is turned into a list of numbers (an embedding) that captures its meaning. Chunks never span two documents; if they did, we could no longer say which document a piece of text came from, and the sentence-level scoring depends on knowing that.

Two details here cost us time before we got them right. The embeddings are scaled to unit length, which makes a simple multiplication equal to cosine similarity and lets us use a fast exact FAISS index instead of an approximate one. And the embedding model has to be handed to each function

explicitly rather than picked up from a shared variable — a stale shared variable silently produced mismatched vector sizes and ruined a whole set of results before we caught it.

Band B — answering a question. The question is embedded once and searched against the index. Some embedding models need a short prefix on the text (e5 expects "query:" on questions and "passage:" on documents); leaving it out does not cause an error, it just quietly makes retrieval worse, which is the kind of failure that is easy to miss. If hybrid retrieval is switched on, the meaning-based and keyword-based result lists are merged using Reciprocal Rank Fusion, which combines positions rather than scores so the two scales never have to be made comparable. A cross-encoder can then re-score a wider shortlist down to the final five. Those five are placed in the prompt in the configured order, and the generator writes the answer at temperature 0 — meaning no randomness, so the same input always gives the same output.

Band C — scoring the answer. This is drawn as a separate band because it is genuinely separate. A second, stronger model reads the question, the context and the answer, and reports which sentences were relevant, which were used, and whether each answer sentence is supported. Before we compute anything, we throw away any sentence labels the judge invented that do not match real retrieved sentences. Without that step a made-up label inflates the score; it once pushed one of our error figures above 1.0, which is arithmetically impossible and is how we found the bug.

Band D — keeping it running. This band does not appear in textbook RAG diagrams and we think it should. Every item in it exists because its absence cost us a completed run: checkpointing to three places, rotating API keys with backoff, repairing malformed judge output, and recording the exact configuration with every result.

4.2 Technical Design Decisions

Several design choices were made before experimentation began in order to ensure that differences in performance could be attributed to pipeline components rather than implementation artifacts.

- **FAISS for Vector Retrieval:** FAISS was selected as the primary vector index because the project operated at a scale where exact nearest-neighbour search was computationally feasible. This allowed us to avoid approximation effects when comparing retrieval strategies.

- **BM25 for Keyword Retrieval:** BM25 was included as a complementary retrieval method because some domains, particularly legal and financial documents, contain domain-specific terminology where exact keyword matching remains valuable.
- **Hybrid Retrieval via Reciprocal Rank Fusion (RRF):** Hybrid retrieval combines dense semantic retrieval with keyword retrieval. Reciprocal Rank Fusion was selected because it merges ranking positions rather than raw scores, eliminating the need to normalise scores from different retrieval systems.
- **Fixed Retrieval Depth (k = 5):** Preliminary experiments indicated that increasing the number of retrieved passages beyond five produced limited improvements in adherence while increasing context size and latency. Therefore, k = 5 was adopted as the project standard.
- **Context Ordering Strategies:** Forward, reverse, and sides ordering strategies were evaluated because previous research has shown that language models may pay different levels of attention to information depending on its position within the prompt. Testing multiple orderings allowed us to measure the effect directly.
- **Deterministic Generation:** All final experiments used temperature = 0 so that identical inputs produced identical outputs, ensuring that differences between configurations reflected architectural changes rather than randomness

4.3 Pipeline Components

Component	What it does	Options we implemented
Chunking	Cuts documents into searchable pieces	whole_doc, sentence windows, sliding word windows, fixed word budgets, large semantic blocks, clause-aware
Embedding	Turns text into vectors for meaning-based search	MiniLM-L6, e5-large-v2, bge-small, FinBERT, legal-BERT
Vector index	Stores and searches vectors	FAISS flat inner-product (exact search)
Keyword index	Word-overlap search over the same chunks	Okapi BM25
Retrieval	Finds the top pieces for a question	Dense, or hybrid via Reciprocal Rank Fusion; k = 5 kept
Reranking	Re-scores a wider shortlist more accurately	cross-encoder/ms-marco-MiniLM-L-6-v2, on or off
Context ordering	Decides where each piece sits in the prompt	forward, reverse, sides

Generation	Writes the answer	llama-3.3-70b-versatile, temperature 0, strict instruction
Judging	Scores the answer against its context	openai/gpt-oss-120b with the verbatim Appendix 7.4 prompt

Table 4 — Pipeline components and the options available for each.

4.4 How the Experiments Were Run

The experimental work ran in two campaigns, and both are reported here because the relationship between them is itself a result.

The first campaign, run over roughly two months, was a stage-wise ablation ladder of 93 experiments. The method is deliberately unclever: fix every component except one, vary that one, keep whichever level scored best, and carry it forward as fixed into the next stage. A ladder of this shape visits the sum of the levels rather than their product — thirteen runs per domain instead of ninety-six — which is what made it affordable on a free Colab quota. It also has the property that every locked choice traces back to one named experiment, so a reviewer can ask why a component was chosen and be shown the run that chose it. Its weakness is structural: having fixed a component at stage one, the ladder can never revisit it, so any configuration that only wins in combination with a later choice is invisible to it. Two such combinations were later found in the grid.

The second campaign, run in the final week, was the full factorial grid described below. It exists because of a problem the ladder could not solve on its own. During the ladder we varied the generator and the judge as part of the search, and in Finance the leading configuration was scored by llama-3.1-8b judging answers written by llama-3.3-70b — a judge weaker than the generator it was grading. Section 5.4 sets out why that arrangement cannot be trusted. Rather than discard the ladder's conclusions, we re-tested them under a fixed protocol: the generator held constant at llama-3.3-70b, every cell judged by the larger openai/gpt-oss-120b, and the retrieval-side factors swept exhaustively.

The locked configurations reported in Chapter 7 therefore come from the factorial campaign, not the ladder. This is a statement about measurement rather than about search: the ladder was a reasonable way to find a pipeline, but only one of the two campaigns measured with an instrument we can defend. The ladder results are retained in full in the project repository, and Appendix F reports both campaigns' leading configurations side by side.

What the pairing showed is worth stating plainly. The two searches agree on every negative result about domain-specific encoders — neither found that BioBERT, legal-BERT or FinBERT wins on its

own merits when substituted into an otherwise unchanged pipeline. They disagree only where components interact, which is exactly where a greedy ladder is known to be blind. And in three of the five domains the ladder's headline AUCROC was higher than the grid's, which on first reading looks like the ladder winning; on inspection it reflects the weaker judge and lower coverage of the earlier runs rather than a better pipeline. A higher number computed from a worse measurement is not a better result. That comparison, more than any individual configuration, is what the two-campaign design bought us.

For each domain we ran a complete grid of 2 embedders $\times$ 4 chunking strategies $\times$ 2 retrieval methods $\times$ 2 reranking settings $\times$ 3 context orderings, which is 96 configurations. Running the identical grid in every domain is what makes the domains comparable: nothing differs between them except the documents and the two candidate embedders.

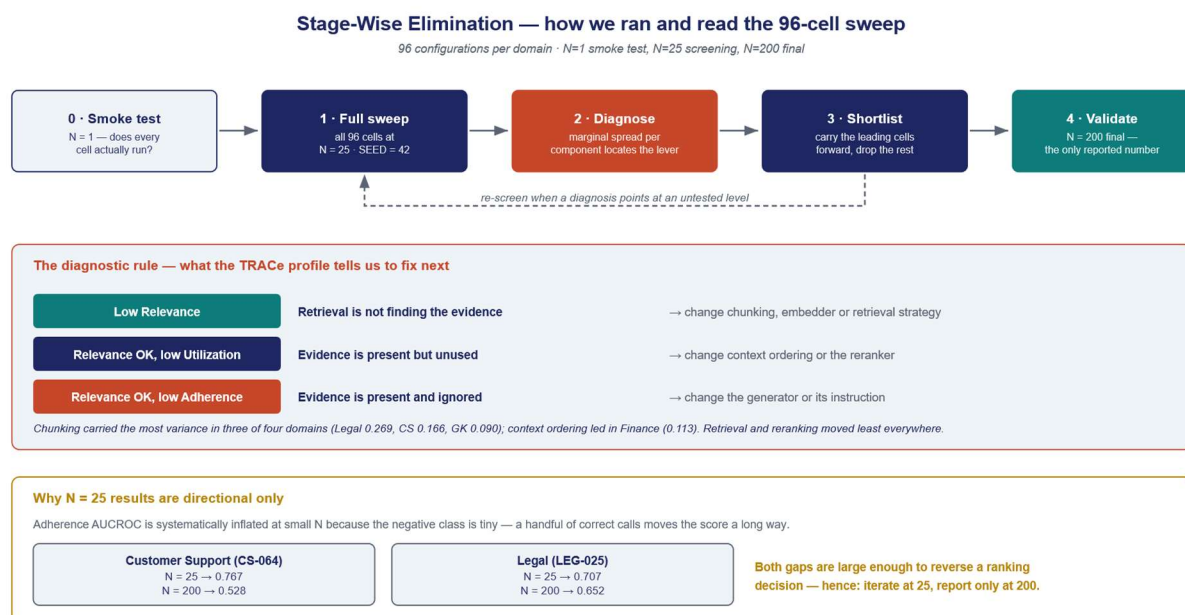


Figure 2 — How each sweep was run and read. We call this stage-wise elimination: screen everything cheaply, work out which stage is carrying the variation, carry only the leading configurations to the expensive final run.

We call the method stage-wise elimination. It is not a clever algorithm; it is a discipline. Run all 96 cells cheaply at 25 examples. Then, instead of just taking the top row, work out how much each individual component moved the score — the difference between the best and worst average for that component across every configuration that used it. That number, the marginal spread, tells you which stage is worth spending the expensive 200-example runs on. Only the shortlisted configurations get a final run.

The TRACe scores are useful here in a way a single quality score would not be, because they localise the failure rather than just reporting it:

What the scores look like	What it means	What to change
Relevance is low	Retrieval is not finding the evidence at all	Chunking, embedding model, or retrieval method
Relevance fine, utilization low	The evidence is there but the model is not using it	Context ordering, or the reranker
Relevance fine, adherence low	The evidence is there and the model is ignoring it	The generator, or the instruction it is given

Table 5 — The diagnostic rule we used to decide what to change next.

4.5 Tools and Frameworks

Purpose	Tool	Why
Runtime	Python 3 on Google Colab	Free, shared across the team, no GPU needed
Data	rungalileo/ragbench on HuggingFace	Supplies both the documents and the reference scores
Embeddings	sentence-transformers	One interface for every model; handles pooling for raw BERT checkpoints
Keyword search	rank-bm25	Standard BM25, nothing to train
Vector search	FAISS (flat, inner product)	Exact search at our scale, so no approximation error to explain away
Reranking	cross-encoder/ms-marco-MiniLM-L-6-v2	Reads question and document together, so it is more accurate than the first-pass search
Model inference	Groq API, with OpenRouter as paid overflow	Fast hosted inference for both writing and judging
Metrics	scikit-learn	RMSE and AUCROC against the reference scores
Interface	Gradio on HuggingFace Spaces	Mentor-specified; quick to build, easy to share
Version control	GitHub (veenulearns-lab)	Also our third backup location
Backup	Google Drive	Survives a Colab restart, unlike local disk

Table 6 — Technology stack and the reason for each choice.

5. Evaluation Metrics

5.1 The Four TRACe Metrics

TRACe splits answer quality into four separate measurements, each answering a different question. All four run at the level of individual sentences, which is what makes them diagnostic rather than just descriptive.

Metric	The question it answers	Formula	Checked against reference using
Context Relevance	Was the material we retrieved actually about the question?	$ \text{relevant} \div \text{all context sentences} $	RMSE — lower is better
Context Utilization	Did the answer actually use that material?	$ \text{utilized} \div \text{all context sentences} $	RMSE — lower is better
Completeness	Did the answer use all of the relevant material?	$ \text{relevant} \cap \text{utilized} \div \text{relevant} $	RMSE — lower is better
Adherence	Is every claim in the answer supported by the documents?	1 if all answer sentences are supported, else 0	AUCROC — higher is better

Table 7 — The four TRACe metrics. RMSE is the average size of the gap between our score and the reference score, so lower is better. AUCROC measures how well our adherence score separates supported answers from unsupported ones; 0.5 is chance and 1.0 is perfect.

5.2 Rules We Follow When Judging

Four rules govern every score in this project. Each one exists because breaking it produced numbers that looked perfectly reasonable and were wrong.

1. The judging prompt from Appendix 7.4 of the RAGBench paper is pasted in by hand, word for word. It is never generated or reworded by a tool. The placeholders are exactly `.format(documents=, question=, answer=)`.
2. Documents are given to the judge as a raw nested list of [key, sentence] pairs, matching the example in the paper. They are never rewritten into flowing prose.
3. Sentence labels returned by the judge are checked against the sentences we actually retrieved, and anything invented is discarded before scoring.
4. Retrieved documents are renumbered 0, 1, 2 and so on in retrieval order before the judge sees them, so the labelling is consistent no matter what order they arrived in.

For adherence we read the judge's overall verdict first and fall back to the sentence-level verdict. RAGBench itself defines adherence as the sentence-level conjunction, so this ordering is our convention rather than the paper's definition. The two only disagree when the judge marks a sentence as not fully supported but still calls the answer supported overall.

5.3 Judge Validation Rationale

This is the rule that changed most during the project, and it is worth explaining properly because it affected every number we report. The reliability of any evaluation framework depends on the quality of the judge performing the assessment. Since TRACe metrics are generated through an LLM-based evaluation process, validating the judge configuration was treated as a critical part of this project rather than as an implementation detail.

Early experiments revealed that changing the judge model could substantially alter reported scores even when the underlying pipeline remained unchanged. This demonstrated that evaluation quality can influence conclusions as much as architectural design choices.

To minimise this risk, the final protocol adopted three principles:

1. Judge Independence

The model generating answers should not evaluate its own output. Independent judging reduces the risk of self-consistent scoring behaviour.

2. Judge Capability

The judge should be at least as capable as, and preferably stronger than, the model being evaluated. A weaker judge may fail to identify subtle hallucinations or unsupported claims.

3. Prompt Fidelity

The original RAGBench judge prompt was reproduced verbatim from the published benchmark. Earlier experiments showed that even minor paraphrasing could significantly alter evaluation outcomes.

Based on these principles, openai/gpt-oss-120b was selected as the final judge and llama-3.3-70b-versatile was used as the answer generator throughout the final experiments. This configuration provides independence, capability, and consistency with the evaluation philosophy used by RAGBench

5.4 The Judge Must Be Stronger Than the Generator

This is the rule that changed most during the project, and it is worth explaining properly because it affected every number we report.

Our early runs used llama-3.3-70b-versatile for both writing and judging, and in some cases a smaller 8b model as judge with the 70b model writing. Both arrangements are wrong, for different reasons. A model grading its own output is not an independent measurement. A weaker model

grading a stronger one cannot reliably tell whether a subtle claim is supported — it lacks the capability to make the judgement it is being asked to make. We confirmed the second point directly: on an identical pipeline, an 8b judge produced an adherence AUCROC of 0.639 where a 70b judge produced 0.875. The pipeline had not changed at all. The instrument had.

The final protocol therefore fixes the judge as openai/gpt-oss-120b and the generator as llama-3.3-70b-versatile. The judge is both independent of the generator and larger than it. This matches how RAGBench itself was built, where GPT-4 graded answers written by weaker models. Every result in Chapters 7 and 9 uses this arrangement.

5.5 Reporting

For every final configuration we report Relevance RMSE, Utilization RMSE and Completeness RMSE (lower is better), and Adherence AUCROC (higher is better), all at 200 examples, next to the RAGBench reference values for the same examples. We also report judge coverage — the proportion of examples the real judge actually scored — because a run where the judge failed on a third of the examples is not comparable to one where it scored everything.

6. Deployment

The deliverable, following mentor guidance, is one application covering every domain rather than one application per domain. A dropdown selects the domain and the app loads that domain's locked configuration.

6.1 Deployment Stack

Part	Technology	Notes
Interface	Gradio	Quick to build, and the mentor-specified framework
Hosting	HuggingFace Spaces	Kept private or access-gated, because a public Space would share our API keys and drain the quota our experiments depend on
Inference	Groq API with key rotation or Open Router Key	Same models as the experiments, so the demo cannot drift from the results
Index	FAISS, built lazily and cached	Cached per chunking and embedding combination so repeat questions do not rebuild it
Programmatic access	FastAPI wrapper	One implementation serving both the demo and the harness

Table 8 — Deployment stack.

6.2 Deployment Flow

1. **Packaging.** Every pipeline stage is a callable module driven by a single dictionary holding each domain's locked configuration.
2. **Interface.** The app provides the five required elements: a domain selector, a question box with sample questions, a Simple versus Advanced toggle comparing the baseline against the tuned configuration, the generated answer with any refusal shown explicitly, and a live TRACe score table.
3. **Traceability.** Each retrieved sentence is marked with a coloured dot showing whether the judge found it relevant, used, or neither. An expandable panel shows the judge's per-sentence reasoning, a timer splits the wait into retrieval, generation and judging, and a note explains that the demo only indexes a sample of the data, so an honest refusal is not a bug.
4. **Hosting.** Deployed as a three-file package — `app.py`, `requirements.txt` and `README.md` — with keys held in Space settings and never committed. We always test locally with `share=True` before pushing.

7. RAGBench: Implementation and Results

Each domain ran the full 96-configuration grid at 25 examples, and the shortlisted configurations were re-run at 200 examples. All runs used `llama-3.3-70b-versatile` to write and `openai/gpt-oss-120b` to judge.

7.1 Which Component Actually Mattered

For each domain we computed the marginal spread of every component — how far apart the best and worst averages are for that component across all configurations that used it. A large spread means the component moved the score; a small one means it barely mattered. We restricted this to runs where the judge scored at least 80% of examples, so that missing judge output could not masquerade as a component effect.

Domain	Chunking	Context order	Embedding	Retrieval	Reranking	Biggest lever
Legal	0.269	0.012	0.074	0.018	0.018	Chunking, by a wide margin
Customer Support	0.166	0.086	0.049	0.049	0.029	Chunking
Finance	0.066	0.113	0.031	0.001	0.007	Context ordering
General Knowledge	0.090	0.002	0.006	0.008	0.022	Chunking

Biomedical	0.127	0.028	0.028	0.001	0.003	Chunking
------------	-------	-------	-------	-------	-------	----------

Table 9 — Marginal spread by component at 25 examples, across all 96 configurations per domain. Legal, Customer Support, Finance and Biomedical are measured on Adherence AUCROC; General Knowledge on Completeness RMSE, because its adherence labels were single-class at this scale and AUCROC cannot be computed.

Three things stand out. Chunking is the largest lever in four of the five domains, and in Legal it dominates everything else combined — which fits, because CUAD contracts are long and structured and how you cut them determines whether a clause survives intact. Context ordering leads only in Finance, where the numbers and the sentences qualifying them need to sit close together. And retrieval and reranking move the score least almost everywhere — in Biomedical their spreads are 0.001 and 0.003, effectively nothing — which was the opposite of what we expected going in; we had assumed that better search would be the main route to better answers.

7.2 Customer Support (TechQA)

Four configurations were promoted to the 200-example run.

Configuration	Rel RMSE ↓	Util RMSE ↓	Comp RMSE ↓	Adherence	AUCROC ↑	Judge coverage
CS-064 · MiniLM · sliding · hybrid · rerank · forward ✓	0.2732	0.1762	0.6179	0.6996	0.5282	0.945
CS-043 · bge-small · fixed 120w · dense · forward	0.4848	0.2950	0.6525	0.7241	0.5022	0.825
CS-037 · bge-small · sliding · dense · forward	0.3357	0.2443	0.5940	0.7474	0.4989	0.820
CS-039 · bge-small · sliding · dense · sides	0.3233	0.2266	0.5963	0.7126	0.4936	0.825

Table 10 — Customer Support final runs, 200 examples. Reference adherence for these examples is 0.605. CS-064 is the locked configuration.

CS-064 wins on every RMSE and on AUCROC, and it also has the highest judge coverage at 94.5%. It is worth noting that it is the only one of the four to use hybrid retrieval and reranking together — a combination we had tested and rejected earlier in the project under the old judge. That reversal is a direct consequence of fixing the judge, and it is a caution against trusting a component decision made with a broken instrument.

The absolute numbers are modest. An AUCROC of 0.528 is only just above chance, and adherence of 0.6996 sits above the reference value of 0.605, meaning our pipeline is producing more supported answers than the benchmark's own reference answers — but our score does not track

the reference labels closely. Section 8.4 explains why we believe that is a property of the labels rather than of the pipeline.

7.3 Legal (CUAD)

Configuration	Rel RMSE ↓	Util RMSE ↓	Comp RMSE ↓	Adherence	AUCROC ↑	Judge coverage
LEG-025 • legal-BERT • sliding • dense • forward ✓	0.2651	0.1346	0.6911	0.8825	0.6524	0.99
LEG-028 • legal-BERT • sliding • dense • sides	0.2651	0.1346	0.6911	0.8825	0.5554	0.99

Table 11 — Legal locked configuration and result.

Legal is our strongest result. Adherence of 0.8825 sits very close to the reference value of 0.905, and the AUCROC of 0.6524 is the best of any domain — meaning our score genuinely distinguishes supported answers from unsupported ones here. It is also the domain where chunking mattered most (spread 0.269), and the winning chunker is a sliding sentence window rather than the clause-aware chunker we built specifically for contracts. We had expected the clause chunker to win; it did not, and we report that rather than quietly dropping it.

7.4 Finance (FinQA)

Configuration	Rel RMSE ↓	Util RMSE ↓	Comp RMSE ↓	Adherence	AUCROC ↑	Judge coverage
FIN-031 • FinBERT • fixed 150w • hybrid • forward ✓	0.3275	0.2815	0.4496	0.8254	0.4480	0.895
FIN-075 • bge-small • fixed 150w • dense • sides	0.3298	0.2621	0.4553	0.7849	0.4492	0.885

Table 12 — Finance final runs, 200 examples. Reference adherence is 0.920. FIN-031 is the locked configuration.

The two configurations split the metrics, so the decision needs a stated rule rather than a glance at the table. FIN-031 wins on adherence (0.8254 against 0.7849), relevance (0.3275 against 0.3298) and completeness (0.4496 against 0.4553); FIN-075 wins on utilization (0.2621 against 0.2815) and on AUCROC (0.4492 against 0.4480). Applying the rule set out in §7.9: FIN-075's AUCROC lead is 0.0012, roughly one twentieth of the 0.02 threshold this report treats as the smallest interpretable difference — it is noise, not a result. FIN-031's adherence lead is 0.0405, thirty-three times larger and well outside that threshold. On the only comparison that clears the noise floor, FIN-031 wins, and it does so on the metric that matters most in a domain where an unsupported

number is worse than a missing one. FinBERT earning its place here is notable, since the specialist embedding models lost in every other domain we tested.

7.5 General Knowledge (HotpotQA)

Configuration	Rel RMSE ↓	Util RMSE ↓	Comp RMSE ↓	Adherence	AUCROC ↑	Judge coverage
GK-031 · e5-large · sliding · hybrid · forward ✓	0.1085	0.0832	0.3782	0.8773	0.6221	0.99
GK-040 · e5-large · fixed 120w · dense · rerank · forward	0.2795	0.2937	0.3845	0.8511	0.6416	0.97

Table 13 — General Knowledge final runs, 200 examples. Reference adherence is 0.915. GK-031 is the locked configuration.

GK-031 gives our best continuous scores anywhere in the project — a relevance RMSE of 0.1085 and a utilization RMSE of 0.0832 mean our predicted scores track the reference values very closely indeed. GK-040 leads on one metric only, AUCROC, by 0.0195. GK-031 leads on the other four: adherence by 0.0262, relevance by 0.1710, utilization by 0.2105 and completeness by 0.0063. Applying the rule in §7.9, GK-040's single advantage sits just under the 0.02 interpretability threshold, while GK-031's relevance and utilization advantages are roughly nine and eleven times that threshold. Trading a 0.171 improvement in relevance error for a 0.0195 improvement in AUCROC would be a poor exchange even if both were equally trustworthy, and §8.4 explains why AUCROC is the less trustworthy of the two here. Hybrid retrieval winning also fits the dataset: HotpotQA questions need evidence from more than one document, and combining keyword and meaning-based search finds more of it.

7.6 Biomedical (CovidQA)

Biomedical ran the same 96-configuration grid as the other four domains, under the same protocol, with llama-3.3-70b-versatile generating and openai/gpt-oss-120b judging. It produced the cleanest measurement conditions of the project: the real judge scored every example of every configuration, giving judge coverage of 1.00 across all 96 cells at the screening stage and all eight configurations carried to 200 examples. Nothing in this domain's numbers is affected by missing judge output.

Configuration	Rel RMSE ↓	Util RMSE ↓	Comp RMSE ↓	Adherence	AUCROC ↑	Judge coverage
BIO-004 · whole_doc · dense · cross-encoder · forward ✓	0.2398	0.1932	0.4605	0.8010	0.6277	1.00

BIO-002 · whole_doc · dense · none · reverse	0.2243	0.1903	0.4808	0.8240	0.6032	1.00
BIO-007 · whole_doc · hybrid · none · forward	0.2314	0.1863	0.4786	0.8270	0.5531	1.00

Table 13a — Biomedical final runs, 200 examples. Eight of the 96 configurations were carried to this scale; the three leading ones are shown. BIO-004 is the locked configuration.

BIO-004 is locked on adherence AUCROC, which is Biomedical's priority metric: CovidQA carries one of the highest hallucination rates in the benchmark, so telling supported answers from unsupported ones matters more here than raw adherence. BIO-004 leads on AUCROC at 0.6277 and has the best completeness error of the domain at 0.4605. BIO-007 reached the highest raw adherence at 0.8270 but a much weaker AUCROC of 0.5531, which is why raw adherence alone was not used to decide; BIO-002 sits between the two on both.

Biomedical also produced the project's sharpest illustration of why screening scores cannot be reported. At 25 examples the leading configuration was BIO-015 with an AUCROC of 0.7727, and BIO-004 — the eventual winner at full scale — screened at just 0.4091, below chance. Had we promoted on the screening ranking alone we would have locked the wrong configuration. At 200 examples BIO-015 falls to 0.5895 and BIO-004 rises to 0.6277. The gap runs in both directions, and it is larger here than anywhere else in the study.

7.7 Summary of Locked Configurations

Domain	Chunking	Embedding	Retrieval	Rerank	Order	Adherence	AUCROC
Customer Support	sliding 5/2	MiniLM-L6	hybrid	cross-encoder	forward	0.6996	0.5282
Legal	sliding 5/2	legal-BERT	dense	none	forward	0.8825	0.6524
Finance	fixed 150w	FinBERT	hybrid	none	forward	0.8254	0.4480
General Knowledge	sliding 5/2	e5-large-v2	hybrid	none	forward	0.8773	0.6221
Biomedical	whole_doc	S-PubMedBert	dense	cross-encoder	forward	0.8010	0.6277

Table 14 — The five locked configurations. Generator llama-3.3-70b-versatile and judge openai/gpt-oss-120b throughout.

The configurations are not identical, but they are less different than we expected. Three of four use a sliding window; three of four use hybrid retrieval; all four use forward ordering; only one uses a reranker. The embedding model is the dimension that varies most, and it varies in the direction the domain suggests — legal-BERT for contracts, FinBERT for financial text, e5-large for open-domain questions, and a small general model for support articles.

So our answer to the generality question is more nuanced than we anticipated. There is real convergence on the mechanical choices — window-based chunking, hybrid search, forward ordering — and real divergence on the embedding model and the chunk size. A team starting fresh could reasonably adopt the shared parts as a default and tune only the embedder and the chunk width per domain.

7.8 Why 25-Example Results Cannot Be Reported

Adherence AUCROC at 25 examples is consistently too optimistic, because the group of unsupported answers is tiny at that scale and a few lucky calls move the score a long way. Two examples from this sweep:

Configuration	AUCROC at N = 25	AUCROC at N = 200	Drop
CS-064 (Customer Support)	0.7667	0.5282	0.239
LEG-025 (Legal)	0.7065	0.6524	0.054

Table 15 — Small-sample inflation. Both drops are real; the Customer Support gap is large enough to have reversed a ranking decision if we had trusted it.

This is why we screen at 25 and report only at 200. It is a slow rule to follow when quota is limited and every 200-example run takes hours, and it was tempting more than once to promote a configuration on its screening score. The Customer Support gap of 0.239 is the reason we did not.

7.9 How a Winner Was Chosen When the Metrics Disagreed

In two domains the leading configurations split the metrics between them: in Finance, FIN-031 led on three measures and FIN-075 on two; in General Knowledge, GK-031 led on four and GK-040 on one. Picking the row with the highest AUCROC would have been simpler, and would have given a different answer in both cases. We did not do that, and the reasoning matters enough to state as a rule rather than leave implicit in each section.

Three principles decide it. First, a difference smaller than roughly 0.02 is not interpretable at our sample size — this report says so in Chapter 11 and again in Chapter 12, and the rule has to apply when it is inconvenient as well as when it is not. Second, when AUCROC and the direct measures disagree, we prefer the direct measures, because §8.4 shows AUCROC is systematically weakened here by the label-transfer effect: it measures agreement with reference labels that describe the benchmark's own answers, not ours, so a pipeline that legitimately retrieves different context is penalised for doing so. Adherence and the three RMSE figures compare our output with the reference values directly and are not distorted in the same way. Third, where the direct

measures still disagree, adherence carries the most weight, because groundedness is the property this project set out to improve.

Applied to the two contested domains, the rule is decisive rather than marginal. FIN-075's AUCROC lead of 0.0012 falls far below the noise floor while FIN-031's adherence lead of 0.0405 sits well above it, so FIN-031 wins on the only comparison that clears the threshold. GK-040's AUCROC lead of 0.0195 sits just under the floor while GK-031's relevance and utilization advantages of 0.1710 and 0.2105 sit far above it, so GK-031 wins on four measures including the two largest effects in the comparison. In both cases the configuration we locked is the one that wins on the measures we can actually trust at this sample size.

We report the runner-up in full in each domain table rather than only the winner, so a reader who prefers AUCROC as the deciding metric can see exactly what the alternative choice would have been and by how much it differs.

8. Challenges, Mistakes and What We Learned

This chapter is here because it contains most of what we actually learned. The hardest problems were not crashes. They were numbers that looked completely reasonable and were wrong, and almost every one was found by accident rather than by testing.

8.1 The Judge Prompt Mattered More Than Any Design Choice

Our early biomedical runs used a reworded version of the RAGBench judging prompt. It said the same thing, produced perfectly well-formed output, and made our relevance error roughly twice as large as it should have been — around 0.44 where the correct prompt gives around 0.22. On the RGB counterfactual test, the same class of substitution moved results by 44.7 percentage points.

That is larger than the effect of any chunking, embedding, retrieval or reranking decision we measured anywhere in the project. It is the single most useful thing we learned, and it is why the judging prompt is now pasted in by a person and never touched by a tool.

8.2 Every Bug We Found and Fixed

#	What went wrong	What it did to the numbers	How we fixed it
1	Judge labels were not checked against the real retrieved sentences	Invented labels inflated the score; one error figure came out above 1.0, which is impossible	Discard any label that does not match a real retrieved sentence

2	Documents were reformatted into prose before being sent to the judge	The judge saw a layout unlike the paper's example and mislabelled its output	Send the raw nested list, exactly as in the paper. Spotted in review by mentor Lokesh
3	Two of the TRACe formulas had their denominators swapped	Utilization and completeness were quietly measuring each other	Corrected both formulas against the paper
4	The RGB counterfactual test used the clean documents instead of the corrupted ones	The counterfactual test contained no counterfactuals at all	Rebuilt the RGB harness from scratch against the correct field
5	An old embedding model was left in a shared variable and reused	Vector sizes silently mismatched, invalidating a whole set of results	Pass the model explicitly into every function; no shared variables
6	Legal contracts contain characters that break JSON parsing	The judge's output failed to parse and runs stopped part-way	Added a repair step and a fallback that recovers whatever fields it can
7	The judging prompt was reworded rather than copied	Relevance error roughly doubled; 44.7-point swing on RGB	Paste the Appendix 7.4 prompt by hand, verbatim
8	Models retired by the provider returned nothing, silently	Runs appeared to finish successfully with empty answers	Check the model exists and fail loudly if a response is empty
9	Answer matching treated "cannot" and "can not" as different	Correct refusals were scored as failures, hiding 29.7 points of performance	Normalise the text before comparing
10	Judge and generator were the same model, or the judge was smaller	Either the model graded itself, or a weaker model judged a stronger one	Judge fixed as gpt-oss-120b, generator as llama-3.3-70b (\$5.3)

Table 16 — Every defect we found. All of them produced numbers. None of them produced an error message.

8.3 Practical Problems and How We Worked Around Them

- **Running out of API quota.** A 200-example run with a large judge can use up a key's daily allowance part-way through. We rotate across six Groq keys with automatic backoff, and when Groq was exhausted we bought OpenRouter credits out of pocket to finish the runs rather than abandon them. This was not in the original plan and is worth flagging for anyone

scoping similar work: hosted inference quota, not compute, was the binding constraint on this entire project.

- **Sessions disconnecting.** We lost several complete runs before we introduced checkpointing. Now every run saves progress after every five examples to three places — the Colab disk, Google Drive and GitHub. That third location seemed excessive until a session started under a different Google account found Drive empty and every result had to be pulled back from GitHub.
- **Models disappearing.** Two of the models we had evaluated were withdrawn by the provider mid-project, and the one we use as our generator is scheduled for retirement shortly after submission. We keep results from withdrawn models, marked as superseded, rather than deleting them.
- **Stale settings in Colab.** A variable set in an earlier cell would still be in memory when we ran a later one, so a run intended for 200 examples would quietly execute at 25 or vice versa. We now print the sample size and pause for three seconds before every long run, so there is time to stop it.

8.4 Why Some AUCROC Scores Sit Close to Chance

Finance (0.4480) and Customer Support (0.5282) have AUCROC scores near 0.5, which normally means a measurement with no discriminating power. We think something more specific is going on, and it is worth setting out because it affects how the whole results table should be read.

The RAGBench reference labels describe whether the benchmark's own answer was supported by the context its annotators saw. Our pipeline retrieves different context and writes a different answer. When it does that well, the reference label stops describing the thing being measured, and the correlation between our score and theirs weakens even as the answers improve. Our clearest evidence is that absolute adherence stays high and sensible — 0.8825 for Legal against a reference of 0.905, 0.8773 for General Knowledge against 0.915 — while the correlation-based metric drifts toward chance.

This is why we report absolute adherence alongside AUCROC everywhere rather than choosing one. It also affects Legal and General Knowledge less than Finance and Customer Support, which is consistent with the explanation: the two domains where our retrieval diverges least from the benchmark's are the two where the labels still fit.

9. RGB: Robustness Evaluation

Everything in Chapter 7 measures how good an answer is when retrieval worked. That leaves the more important production question unanswered: what does the system do when retrieval goes wrong? RGB is the benchmark built for exactly that, and this chapter covers it end to end.

9.1 What RGB Tests, and Why It Matters

A deployed RAG system will routinely be handed irrelevant documents, incomplete evidence, evidence split across several sources, and occasionally documents that are simply wrong. RGB tests each of those conditions in isolation, which means a failure points at a specific weakness rather than a general one. The four abilities and how they are scored:

Ability	What it tests	How it is measured	Formula
Noise robustness	Can the model still find the answer when irrelevant documents are mixed in?	Accuracy at increasing noise ratios (0 to 0.8)	$\text{Acc} = \text{correct answers} \div \text{total questions, at each noise level}$
Negative rejection	Does the model refuse when the evidence genuinely is not there?	Rejection rate on questions with no supporting document	$\text{Rej} = \text{refusals} \div \text{total unanswerable questions}$
Information integration	Can the model combine evidence from several documents?	Accuracy on questions requiring two or more sources	$\text{Acc} = \text{correct answers} \div \text{total multi-source questions}$
Counterfactual robustness	Does the model notice when a retrieved document is factually wrong?	Answer accuracy, error detection and error correction	$\text{Acc_doc} = \text{errors correctly identified} \div \text{total corrupted documents}$; $\text{ED} = \text{detection rate}$; $\text{CR} = \text{corrections} \div \text{detections}$

Table 17 — The four RGB abilities and their formulas.

9.2 Dataset

RGB provides 300 English question-answer pairs per ability, drawn from news articles, along with a matching Chinese set. We evaluated the English set only. Three files are used: the base set, an information-integration set built from questions needing multiple sources, and a counterfactual set in which a fact inside a retrieved document has been deliberately altered. Noise is created by mixing controlled proportions of irrelevant passages into the retrieved documents.

9.3 Methodology

Our first RGB campaign (v1) evaluated four models and had to be discarded. The reason is instructive: the counterfactual test had been reading the dataset's clean-document field rather

than the corrupted one, so the test that was supposed to plant factual errors contained no errors at all. Every model passed it, which we had briefly taken as a good sign.

We rebuilt the harness from scratch as v2, fixing three defects from v1, and re-ran everything. The v2 method:

1. Load the correct dataset field for each condition, verified by inspecting individual examples before running the batch rather than after.
2. Run noise robustness and negative rejection at 300 examples, and information integration and counterfactual robustness at 100.
3. Rotate across nine Groq API keys with a halt-guard that stops the run if error responses start appearing, so a provider problem does not silently fill the results with blanks.
4. Apply a single rescoring pass at the end with text normalisation, so that formatting differences in a refusal are not counted as wrong answers.
5. Checkpoint to three locations, as in the RAGBench work.

9.4 Results

Model	Noise 0 → 0.8	Negative rejection	Information integration	Counterfactual Acc / Acc_doc
llama-3.3-70b-versatile	100 / 100 / 99.3 / 99.3 / 99.0	67.3 normalised · 37.7 strict	80 / 79 / 74	93 / 61
qwen2.5-72b	99.7 / 99.7 / 99 / 99 / 96	73.0	78 / 76 / 71	92 / 71
openai/gpt-oss-120b	98 / 98 / 98 / 98.3 / 94.7	71.7	75 / 72 / 66	91 / 51
openai/gpt-oss-20b	97.3 / 98 / 97 / 97.3 / 95	70.3	70 / 66 / 63	71 / 26
llama-3.1-8b-instant	99.7 / 99.3 / 98.3 / 96.7 / 86	74 normalised · 63.7 strict	68 / 64 / —	—

Table 18 — RGB v2 results. Noise columns are accuracy at noise ratios 0, 0.2, 0.4, 0.6 and 0.8. 300 examples for noise and rejection, 100 for integration and counterfactual.

9.5 What the RGB Results Show

- **Noise is not the problem we expected.** Every model handles noise well until the very highest setting, and even then the larger ones barely degrade. The 8b model is the exception, dropping to 86 at noise 0.8 where the 70b model holds at 99. If your index is noisy, model size buys you real robustness.

- **Spotting wrong documents is where models differ most.** Detecting that a retrieved document is factually wrong falls away sharply with model size: 71 for qwen2.5-72b, 61 for llama-3.3-70b, 51 for gpt-oss-120b, and 26 for gpt-oss-20b. This is the ability that matters most for any deployment whose index might contain stale or incorrect documents, and it is the one that survives compression least well.
- **Refusal is the weakest ability across the board.** Refusal rates sit between 67 and 74 under a normalised scoring rule and much lower under a strict one — 37.7 against 67.3 for the 70b model. We report both, because reporting only the flattering figure would give a seriously misleading picture of how often the system correctly declines.
- **Answer matching is part of the instrument.** The difference between "cannot" and "can not" was hiding 29.7 points of performance on reasoning-style models until we normalised the text. How you compare answers is part of the measuring instrument and needs checking like everything else.

9.6 Challenges in the RGB Work

- **A test that tested nothing.** The v1 counterfactual test read the wrong field and contained no counterfactuals. We only found it because the results were suspiciously good across every model. The lesson we took is to inspect a handful of actual examples from a new test set before running the batch, not after.
- **Two providers, one benchmark.** RGB across five models is many thousands of API calls. Groq's daily limits stopped us repeatedly, and we bought OpenRouter credits to complete the qwen2.5-72b and llama-3.1-8b runs. Splitting a benchmark across two providers means checking that the same model on both routes gives the same answers before combining results.
- **Models retired mid-campaign.** Two models in our v1 set were withdrawn by Groq partway through. We kept the v1 numbers, marked superseded, and made sure at least one model appeared in both v1 and v2 so the two campaigns could be related to each other.
- **Prompt fidelity, again.** As with TRACe, using the paper's exact prompt rather than a reasonable paraphrase was worth up to 44.7 points. Two different benchmarks, the same lesson.

10. Impact and Industry Relevance

- **Customer Support:** Faster ticket resolution, reduced agent effort, and more consistent responses through retrieval of verified support documentation.
- **Legal:** Improved contract discovery, clause retrieval, and document review efficiency by enabling grounded search across large legal documents.

- Finance: Faster access to supporting evidence for financial analysis, helping users locate relevant figures and the contextual information needed to interpret them correctly.
- Healthcare/Biomedical: Better evidence-grounded information retrieval, reducing the risk of unsupported responses in knowledge-intensive domains.
- General Knowledge: Improved knowledge discovery by combining evidence from multiple sources, supporting research, education, and enterprise knowledge-management use cases.

The project demonstrates how Retrieval-Augmented Generation (RAG) can reduce hallucinations, improve answer traceability, and increase user confidence by grounding responses in retrieved evidence rather than relying solely on model memory.

11. Future Scope

11.1 Finishing What Is Started

- Promote more Biomedical configurations to the 200-example stage. Eight of 96 were run at final scale, and the screening leader was not among the strongest of them.
- Promote more configurations to the 200-example stage. Legal has only one final run, which means we cannot say how far ahead of second place it is.

11.2 Making the Numbers More Trustworthy

Every figure in this report is a single point estimate. Confidence intervals computed by resampling would tell us which differences are real and which are noise, and would directly address the Finance case where two configurations are separated by 0.001 on AUCROC. Until that is done, differences smaller than roughly 0.02 should not be treated as meaningful.

11.3 Testing the Label Explanation

Our explanation for the near-chance AUCROC scores in §8.4 is reasoned but not measured. The clean test is to hold retrieval fixed at exactly the context the benchmark's annotators saw, generate with our pipeline, and re-score. If AUCROC recovers, the explanation is confirmed. This is the single most valuable experiment left undone.

11.4 Improving the Instruction Rather Than the Model

Since the prompt turned out to be the largest lever we found, the cheapest remaining improvement is instruction design — particularly around refusing cleanly, which RGB shows is the weakest ability across every model we tested.

11.5 Longer-Term Directions

- Retriever fine-tuning targeted at the RGB abilities — training with deliberately noisy examples to improve noise robustness, and with negative examples to improve refusal. This was in our original proposal and was descope after discussion with mentors because we had no GPU access.
- Query rewriting and decomposition, evaluated as swept options rather than adopted because they are fashionable.
- Chunking strategies learned from the corpus, given that chunking was the largest lever in four of five domains.
- Extension to messy enterprise sources such as email, chat and ticketing systems, where provenance matters more and documents are far less clean than a benchmark.
- Running everything on a local model to remove the API quota that constrained this project more than anything else.

12. Threats to Validity

Every empirical study has limitations, and this project is no exception. The following factors should be considered when interpreting the reported results.

Benchmark Dependence: Results were evaluated primarily using RAGBench and RGB datasets. Performance on these benchmarks may not fully represent behaviour in real-world enterprise environments.

Sample Size Constraints: Final evaluations were conducted on approximately 200 examples per domain. Although significantly more reliable than the 25-example screening runs, larger evaluation sets may produce slightly different rankings.

LLM Judge Dependence: The project relies on an LLM-based evaluation framework. While substantial effort was spent validating the judge configuration, automated evaluation may still differ from human assessment in some cases.

Label Transfer Effect: RAGBench reference labels describe the benchmark's original answers rather than the answers generated by our pipeline. As retrieval and generation behaviour diverges from the original benchmark system, correlation-based metrics such as AUCROC may become less informative.

Infrastructure Constraints: Experiments were conducted using hosted inference services operating under quota limits and session restrictions. Operational constraints occasionally affected experiment scheduling and required checkpointing and recovery mechanisms.

Despite these limitations, the use of consistent protocols, fixed sampling procedures, benchmark reference labels, and repeated validation checks provides confidence that the reported findings accurately reflect the relative behaviour of the evaluated pipeline configurations.

13. Conclusion

We set out to find which part of a RAG pipeline is actually responsible for answer quality, whether the answer is the same across domains, and whether our own measurements could be trusted. The third question turned out to be the hardest and the most instructive.

On attribution, chunking carried the most variation in four of the five domains and was overwhelmingly dominant in Legal, where a 0.269 spread dwarfed everything else. Context ordering led only in Finance. Retrieval method and reranking — the two components we had assumed would matter most when we started — moved the score least almost everywhere, and in Biomedical they were essentially flat at 0.001 and 0.003. That reversal of expectations is the clearest single lesson of the project for us.

On generality, the answer is more balanced than we anticipated. The four locked configurations converge on window-based chunking, forward ordering and mostly hybrid retrieval, and diverge on the embedding model, which varies in the direction each domain suggests. A sensible default exists; the embedder and chunk width still need local tuning.

On validity, the findings were uncomfortable and are the ones we would most want carried forward. The largest single lever we measured anywhere was not an architectural decision at all — it was copying the judging prompt word for word instead of rewording it, worth up to 44.7 points and roughly halving our relevance error. Ten defects produced confident, well-formatted, wrong numbers: invented labels pushing an error figure above 1.0, two formulas measuring each other, a counterfactual test containing no counterfactuals, an old model left in a variable, and a judge either grading its own output or too small to make the judgement asked of it. Every one of them would have survived a test that only asked whether the code ran.

We came to this project without technical backgrounds and learned the subject by building it. In hindsight that was an advantage in one specific way: we did not assume our measurements were correct, and most of what we found came from following up numbers that looked wrong. If there is one thing we would tell the next team, it is that RAG evaluation code fails quietly rather than loudly, and the time spent checking the instrument is better spent than the time spent tuning the pipeline.

14. References

- [1] P. Lewis, E. Perez, A. Piktus, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," NeurIPS, 2020.
- [2] R. Friel, M. Belyi, and A. Sanyal, "RAGBench: Explainable Benchmark for Retrieval-Augmented Generation Systems," arXiv:2407.11005, 2024. <https://arxiv.org/pdf/2407.11005>
- [3] J. Chen, H. Lin, X. Han, and L. Sun, "Benchmarking Large Language Models in Retrieval-Augmented Generation," AAAI, 2024. arXiv:2309.01431. <https://arxiv.org/pdf/2309.01431>
- [4] X. Wang, Z. Wang, X. Gao, et al., "Searchahing for Best Practices in Retrieval-Augmented Generation," EMNLP, 2024.
- [5] V. Karpukhin, B. Oğuz, S. Min, et al., "Dense Passage Retrieval for Open-Domain Question Answering," EMNLP, 2020.
- [6] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," Foundations and Trends in Information Retrieval, 2009.
- [7] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods," ACM SIGIR, 2009.
- [8] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," EMNLP-IJCNLP, 2019.
- [9] N. F. Liu, K. Lin, J. Hewitt, et al., "Lost in the Middle: How Language Models Use Long Contexts," TACL, vol. 12, 2024.
- [10] R. Nogueira and K. Cho, "Passage Re-ranking with BERT," arXiv:1901.04085, 2019.
- [11] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," IEEE Transactions on Big Data, 2021.
- [12] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," EACL (Demonstrations), 2024.
- [13] Y. Gao, Y. Xiong, X. Gao, et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," arXiv:2312.10997, 2023.
- [14] RAGBench dataset: <https://huggingface.co/datasets/rungalileo/ragbench>

Appendix A — Settings Held Constant

Setting	Value	Why
Random seed	42	Same examples in every experiment, so results are comparable
Screening size	25 examples	Fast enough to sweep 96 configurations; directional only
Final size	200 examples	The only scale at which we report a result
Chunks kept	5	Larger values gave no adherence gain in our k-sweeps
API timeout	30 seconds	Stops a hung call without discarding slow but valid ones
Checkpoint interval	Every 5 examples	Saved to Colab disk, Google Drive and GitHub
Generator	llama-3.3-70b-versatile, temperature 0	Deterministic, so runs are repeatable
Judge	openai/gpt-oss-120b	Independent of the generator and larger than it
Judge prompt	Appendix 7.4, pasted by hand	Rewording it cost up to 44.7 points
Judge coverage floor	80%	Below this a run is quarantined, not reported

Table A.1 — Settings held constant across every domain and experiment.

Appendix B — Environment and Secrets

Variable	Purpose
GROQ_API_KEY_1 ... GROQ_API_KEY_6	Rotating Groq credentials, to spread usage across daily limits
OPENROUTER_API_KEY	Paid overflow capacity when Groq quota was exhausted
HF_TOKEN	HuggingFace dataset and model downloads (must be uppercase)
HF_HUB_ENABLE_HF_TRANSFER=0	Disables the fast-transfer path that caused download stalls in Colab
GIT_PAT_V / GH_PAT_NEW	GitHub access token, our third backup location
Drive path	/content/drive/MyDrive/RAGBench_Results

Table B.1 — Runtime configuration.

Appendix C — Reproducing a Result

Every artefact referenced in this report — the pipeline scripts, the executed notebooks, the results files for both campaigns, and the RGB harness — is in the public project repository at <https://github.com/veenulearns-lab/RAGBench-Capstone-Batch26>. The steps below assume a clone of it.

```
# 1. Open a domain notebook, e.g. RAGBench/notebooks/legal_greedy_01_final.ipynb
# 2. Confirm Colab secrets: GROQ_API_KEY_1..6, HF_TOKEN, GIT_PAT_V
# 3. Paste the Appendix 7.4 judge prompt into Cell 8, by hand:
#   JUDGE_PROMPT_TEMPLATE = """"PASTE_PAPER_PROMPT_HERE""""
# 4. Run the single-example check before any batch run
# 5. Set the size and confirm the guard print:
#     N_SAMPLES = 200
#     print(f'*** N_SAMPLES={N_SAMPLES} ***'); time.sleep(3)
# 6. Run the harness. It saves every 5 examples to three locations
#    and resumes automatically if the session drops.
# 7. Results are pushed to RAGBench/results/factorial/<domain>/ in the repo
```

Appendix D — Judge Prompt (Verbatim, Appendix 7.4)

The prompt below is reproduced word for word from Appendix 7.4 of Friel, Belyi and Sanyal (2024), "RAGBench: Explainable Benchmark for Retrieval-Augmented Generation Systems" (arXiv:2407.11005). It is pasted in by hand and is never generated, reworded or edited by a tool. Section 8.1 records what happened when an earlier version of this project used a reworded prompt instead: our relevance error roughly doubled, and RGB results moved by 44.7 percentage points. The text is therefore treated as a fixed artefact of the benchmark rather than as something the project may restate.

The template is called with exactly three named placeholders. Documents are passed in raw nested `[[key, sentence], ...]` form, matching the worked example in the paper; they are never reformatted into prose.

```
JUDGE_PROMPT_TEMPLATE = """"
```

```
    I asked someone to answer a question based on one or more
documents. Your task is to review their response and assess whether or not each
sentence in that response is supported by text in the documents. And if so, which
sentences in the documents provide that support. You will also tell me which
of the documents contain useful information for answering the question, and
which of the documents the answer was sourced from.
```

```
Here are the documents, each of which is split into sentences. Alongside each
sentence is associated key, such as '0a.' or '0b.' that you can use to refer
to it:
```

```
```\n{documents}\n```
```

The question was:

```
```\n{question}\n```
```

Here is their response, split into sentences. Alongside each sentence is associated key, such as 'a.' or 'b.' that you can use to refer to it. Note that these keys are unique to the response, and are not related to the keys in the documents:

```
```\n{answer}\n```
```

You must respond with a JSON object matching this schema:

```
{{\n "relevance_explanation": string,\n "all_relevant_sentence_keys": [string],\n "overall_supported_explanation": string,\n "overall_supported": boolean,\n "sentence_support_information": [\n {{\n "response_sentence_key": string,\n "explanation": string,\n "supporting_sentence_keys": [string],\n "fully_supported": boolean\n }}\n],\n "all_utilized_sentence_keys": [string]\n}}
```

The `relevance_explanation` field is a string explaining which documents contain useful information for answering the question. Walk through the information in the documents step by step and how it is useful for answering the question.

The `all_relevant_sentence_keys` field is a list of all document sentence keys (e.g. '0a') that are relevant to the question. Include every sentence that is useful and relevant to the question, even if it was not used in the

response, or if only parts of the sentence are useful. Base this judgement only on the documents and the question -- ignore the response entirely when deciding relevance. Leave out sentences that could be removed from the document without affecting someone's ability to answer the question.

The `overall_supported_explanation` field is a string explaining why the response *as a whole* is or is not supported by the documents. Walk through each claim in the response individually and assess its support (or lack of support) in the documents one at a time, before drawing any conclusion about the response as a whole.

The `overall_supported` field is a boolean reflecting the conclusion you reached at the end of `overall_supported_explanation`: whether the response as a whole is supported by the documents.

The `sentence_support_information` field is a list of objects, one for each sentence in the response. Each object **MUST** have the following fields:

- `response_sentence_key`: a string identifying the sentence in the response. This key is the same as the one used in the response above.
- `explanation`: a string explaining why the sentence is or is not supported by the documents.
- `supporting_sentence_keys`: keys (e.g. '0a') of sentences from the documents that support the response sentence. If the sentence is not supported, this list **MUST** be empty. If the sentence is supported, this list **MUST** contain one or more keys. In special cases where the sentence is supported, but not by any specific sentence, you can use the string "supported\_without\_sentence" to indicate that the sentence is generally supported by the documents. Consider cases where the sentence is expressing inability to answer the question due to lack of relevant information in the provided context as "supported\_without\_sentence". In cases where the sentence is making a general statement (e.g. outlining the steps to produce an answer, or summarizing previously stated sentences, or a transition sentence), use the string "general". In cases where the sentence is correctly stating a well-known fact, like a mathematical formula, use the string "well\_known\_fact". In cases where the sentence is performing numerical reasoning (e.g. addition, multiplication), use the string "numerical\_reasoning".
- `fully_supported`: a boolean indicating whether the sentence is fully supported by the documents.
  - This value should reflect the conclusion you drew at the end of your step-by-step breakdown in explanation.
  - If `supporting_sentence_keys` is an empty list, then `fully_supported` must be false.
  - Otherwise, use `fully_supported` to clarify whether everything in the response sentence is fully supported by the document text indicated in `supporting_sentence_keys` (`fully_supported` = true), or whether the sentence is only partially or incompletely supported by that document text (`fully_supported` = false).

The `all_utilized_sentence_keys` field is a list of all sentences keys (e.g. '0a') that were used to construct the answer. Include every sentence that either directly supported the answer, or was implicitly used to construct the answer, even if it was not used in its entirety. Omit sentences that were not used, and could have been removed from the documents without affecting the answer.

You must respond with a valid JSON string. Use escapes for quotes, e.g. `'\"'`, and newlines, e.g. `'\\n'`. Do not write anything before or after the JSON string. Do not wrap the JSON string in backticks like `“` or `“json`.

As a reminder: your task is to review the response and assess which documents contain useful information pertaining to the question, and how each sentence in the response is supported by the text in the documents.

"""

```
prompt = JUDGE_PROMPT_TEMPLATE.format(
 documents = str(documents_sentences), # raw nested [[key, sentence], ...]
 question = question,
 answer = answer,
)
```

Judge model: openai/gpt-oss-120b. Generator under test: llama-3.3-70b-versatile. The judge is both independent of the generator and larger than it, matching the arrangement RAGBench itself used when the reference labels were created.

## Appendix E — Scoring table (csv)- Header explanation

Header	Domains Used	Meaning	Calculation / Logic
exp_id	All	Experiment ID	Unique identifier string (e.g., CS-001, FIN-006)
timestamp	All	Time of run	Auto-logged when experiment executed
domain	All	Dataset type	Finance, Legal, Customer Support, GK
status	All	Run status	Done / error
embedder	All	Embedding model	e.g., finbert, e5-large-v2
chunk_config	All	Chunking strategy	whole_doc, sliding_5o2, sentence_3o1, fixed_120w
gen_model	All	Generator model	e.g., llama-3.3-70b
judge_model	All	Judge model	e.g., gpt-oss-120b
retrieval	All	Retrieval type	dense / hybrid / sparse
rerank	All	Reranker used	none / cross_encoder

context_order	All	Order of context	forward / reverse / sides
k_retrieve	All	Retrieval depth	Number of docs initially retrieved
k_final	All	Final context size	Number of docs passed to generator
seed	All	Random seed	Integer controlling reproducibility
n_examples	All	Sample size	Number of examples tested (e.g., 25)
context_relevance	All	Relevance score	Judge score: retrieved context vs query
context_utilization	All	Utilization score	Judge score: how much of context was used
completeness	All	Completeness score	Judge score: coverage of gold answer
adherence	All	Adherence score	Judge score: alignment with gold answer
ref_context_relevance	All	Reference relevance	Gold/reference score for context relevance
ref_context_utilization	All	Reference utilization	Gold/reference score for context usage
ref_completeness	All	Reference completeness	Gold/reference score for completeness
ref_adherence	All	Reference adherence	Gold/reference score for adherence
context_relevance_rmse	All	Error metric	Root Mean Square Error vs gold relevance
context_utilization_rmse	All	Error metric	RMSE vs gold utilization
completeness_rmse	All	Error metric	RMSE vs gold completeness
adherence_rmse	All	Error metric	RMSE vs gold adherence
context_relevance_mae	All	Error metric	Mean Absolute Error vs gold relevance
context_utilization_mae	All	Error metric	MAE vs gold utilization
completeness_mae	All	Error metric	MAE vs gold completeness
adherence_mae	All	Error metric	MAE vs gold adherence
adherence_auroc	All	AUROC	Area under ROC curve for adherence classification
adherence_accuracy	All	Accuracy	Correct adherence predictions / total
adherence_pos_rate	All	Positive rate	Proportion of positive adherence cases
n_gold_adherence	All	Gold labels count	Number of gold adherence labels
judge_used	All	Judge coverage	Fraction of examples judged
runtime_sec	All	Runtime	Seconds taken for run

## Appendix F — The Two Campaigns, and the Leading Grid Configurations

This appendix supports §4.4. Table F1 places each domain's greedy ladder winner next to its factorial grid winner. Tables F2–F6 list the eight highest-scoring cells of the 96-configuration grid in each domain at the 25-example screening scale, restricted to cells where the judge scored at least 80% of examples. These are screening figures and are not reported as results; §7.8 explains why.

They are included so that the shortlist carried to the 200-example runs can be checked against the grid it came from.

Domain	Greedy winner	Factorial winner	Greedy AUCROC	Grid AUCROC	Where they differ
Biomedical	EXP-020 · fixed-200w · S-PubMedBERT · dense · cross-encoder · sides	BIO-004 · whole_doc · S-PubMedBert · dense · cross-encoder · forward	0.6587 (n=217)	0.6277 (n=200)	Agreement. Same encoder, same reranker; only the chunker differs.
Customer Support	CS-EXP-FINAL · sliding 120w/40 · MiniLM · dense · none · forward	CS-064 · sliding_5o2 · MiniLM · hybrid · cross-encoder · forward	0.6430 (n=200)	0.5282 (n=200)	Same encoder. The ladder used gpt-oss-120b as generator; the grid fixed it at 70b.
Legal	LEGAL-EXP-FINAL · sentence 3s/1o · MiniLM · dense · none · forward	LEG-025 · sliding_5o2 · legal-BERT · dense · none · forward	0.5334 (n=200)	0.6524 (n=200)	The clearest disagreement: the grid's best cell uses legal-BERT, which the ladder rejected.
General Knowledge	EXP-FINAL-BEST · whole_doc · e5-large-v2 · hybrid · cross-encoder · forward	GK-040 · fixed_120w · e5-large-v2 · dense · cross-encoder · forward	0.4866 (n=199)	0.6416 (n=200)	Same encoder and reranker. AUCROC is near-blind here (see §8.4).
Finance	NEW-A · whole_doc · FinBERT · dense k=3 · none · sides	FIN-031 · fixed_150w · FinBERT · hybrid · none · forward	0.6196 (n=196)	0.4480 (n=200)	Both keep FinBERT. The ladder's final run was judged by llama-3.1-8b, weaker than its generator.

*Table F1 — Greedy ladder winner against factorial grid winner, by domain. The AUCROC columns are not directly comparable: the ladder varied generator and judge as part of its search, and in Finance the final ladder run was judged by a model weaker than its own generator.*

Three of the five ladder winners post a higher AUCROC than the corresponding grid winner. That gap is not evidence that the ladder found a better pipeline. In every such case the ladder run was scored under a different judge, at a different coverage, or both, so the two numbers are measurements of different things rather than two measurements of the same thing.

Exp ID	Chunking	Embedder	Retrieval	Rerank	Order	Rel ↓	Util ↓	Comp ↓	Adher ↑	AUCROC ↑
BIO-015	sent 4o1	S-PubMedBert	dense	none	sides	0.2035	0.0894	0.4410	0.813	0.7727
BIO-007	whole doc	S-PubMedBert	hybrid	none	fwd	0.2121	0.1612	0.4351	0.835	0.7348
BIO-020	sent 4o1	S-PubMedBert	hybrid	none	rev	0.2379	0.1373	0.5312	0.740	0.7348
BIO-021	sent 4o1	S-PubMedBert	hybrid	none	sides	0.2253	0.1002	0.4818	0.780	0.7121
BIO-063	sent 4o1	MiniLM-L6-v2	dense	none	sides	0.2198	0.1228	0.4900	0.944	0.6515
BIO-067	sent 4o1	MiniLM-L6-v2	hybrid	none	fwd	0.1757	0.1211	0.4825	0.880	0.6212
BIO-028	slide 5o2	S-PubMedBert	dense	cross-enc	fwd	0.2391	0.1302	0.4873	0.885	0.6136
BIO-076	slide 5o2	MiniLM-L6-v2	dense	cross-enc	fwd	0.2391	0.1302	0.4873	0.885	0.6136

*Table F2 — Biomedical (CovidQA): the eight highest-AUCROC grid cells at 25 examples. Judge coverage was 1.00 in every cell, the cleanest measurement conditions of any domain.*

Exp ID	Chunking	Embedder	Retrieval	Rerank	Order	Rel ↓	Util ↓	Comp ↓	Adher ↑	AUCROC ↑
CS-064	slide 5o2	MiniLM-L6-v2	hybrid	cross-enc	fwd	0.2615	0.2233	0.4767	0.5969	0.7667
CS-009	sent 3o1	MiniLM-L6-v2	dense	none	sides	0.4048	0.1989	0.5468	0.4687	0.7067
CS-015	slide 5o2	MiniLM-L6-v2	dense	none	sides	0.2526	0.1591	0.4682	0.7006	0.6833
CS-003	whole doc	MiniLM-L6-v2	dense	none	sides	0.0709	0.0298	0.4981	0.5810	0.6767
CS-051	whole doc	MiniLM-L6-v2	hybrid	none	sides	0.0916	0.0799	0.5052	0.5564	0.6767
CS-087	slide 5o2	bge-small	hybrid	none	sides	0.3169	0.1687	0.5613	0.6803	0.6733
CS-018	slide 5o2	MiniLM-L6-v2	dense	cross-enc	sides	0.2660	0.1919	0.5705	0.6446	0.6467
CS-054	whole doc	MiniLM-L6-v2	hybrid	cross-enc	sides	0.0597	0.0381	0.5796	0.5321	0.6400

*Table F3 — Customer Support (TechQA): the eight highest-AUCROC grid cells at 25 examples. CS-064 leads the screen and was subsequently locked, but its AUCROC fell from 0.767 to 0.528 at 200 examples — see Table 15.*

Exp ID	Chunking	Embedder	Retrieval	Rerank	Order	Rel ↓	Util ↓	Comp ↓	Adher ↑	AUCROC ↑
FIN-032	fixed 150w	finbert	hybrid	none	rev	0.3448	0.2707	0.5783	0.938	0.7024
FIN-020	sent 4o1	finbert	hybrid	none	rev	0.2735	0.2237	0.5937	0.859	0.6607
FIN-016	sent 4o1	finbert	dense	cross-enc	fwd	0.1358	0.1152	0.4277	0.816	0.6548

FIN-022	sent 4o1	finbert	hybrid	cross-enc	fwd	0.1662	0.1498	0.3208	0.816	0.6548
FIN-026	fixed 150w	finbert	dense	none	rev	0.2985	0.2722	0.5142	0.851	0.6429
FIN-035	fixed 150w	finbert	hybrid	cross-enc	rev	0.3430	0.2608	0.5963	0.883	0.6012
FIN-029	fixed 150w	finbert	dense	cross-enc	rev	0.3430	0.2608	0.5963	0.883	0.6012
FIN-050	whole doc	bge-small	dense	none	rev	0.0573	0.0604	0.4362	0.784	0.5952

*Table F4 — Finance (FinQA): the eight highest-AUCROC grid cells at 25 examples. Seven of the eight use FinBERT, the only domain in which a specialist encoder dominates the leading cells.*

Exp ID	Chunking	Embedder	Retrieval	Rerank	Order	Rel ↓	Util ↓	Comp ↓	Adher ↑	AUCROC ↑
LEG-025	slide 5o2	legal-bert	dense	none	fwd	0.2500	0.0944	0.6754	0.8800	0.7065
LEG-027	slide 5o2	legal-bert	dense	none	sides	0.2364	0.0934	0.6899	0.8800	0.7065
LEG-028	slide 5o2	legal-bert	dense	cross-enc	fwd	0.2316	0.0676	0.7049	0.8640	0.6957
LEG-030	slide 5o2	legal-bert	dense	cross-enc	sides	0.2241	0.0629	0.6994	0.8433	0.6848
LEG-026	slide 5o2	legal-bert	dense	none	rev	0.2479	0.1191	0.6720	0.8200	0.6739
LEG-029	slide 5o2	legal-bert	dense	cross-enc	rev	0.2132	0.0566	0.6794	0.8227	0.6739
LEG-035	slide 5o2	legal-bert	hybrid	cross-enc	rev	0.2268	0.0697	0.5901	0.8133	0.6630
LEG-036	slide 5o2	legal-bert	hybrid	cross-enc	sides	0.2400	0.0743	0.6694	0.8000	0.6630

*Table F5 — Legal (CUAD): the eight highest-AUCROC grid cells at 25 examples. All eight use legal-BERT and sliding\_5o2, and judge coverage was 1.00 throughout; the ladder had rejected legal-BERT when substituted on its own.*

Exp ID	Chunking	Embedder	Retrieval	Rerank	Order	Rel ↓	Util ↓	Comp ↓	Adher ↑	Judge cov.
GK-007	whole doc	e5-large-v2	hybrid	none	fwd	0.0866	0.0605	0.2589	0.90	0.92
GK-037	fixed 120w	e5-large-v2	dense	none	fwd	0.2514	0.2571	0.2799	0.90	1.00
GK-063	sent 3o1	MiniLM-L6-v2	dense	none	sides	0.1042	0.0786	0.2860	0.82	0.92
GK-001	whole doc	e5-large-v2	dense	none	fwd	0.0875	0.0691	0.2888	0.94	0.96
GK-040	fixed 120w	e5-large-v2	dense	cross-enc	fwd	0.2514	0.2622	0.2972	0.94	1.00
GK-046	fixed 120w	e5-large-v2	hybrid	cross-enc	fwd	0.2514	0.2622	0.2972	0.94	1.00
GK-094	fixed 120w	MiniLM-L6-v2	hybrid	cross-enc	fwd	0.2514	0.2622	0.2972	0.94	1.00
GK-085	fixed 120w	MiniLM-L6-v2	dense	none	fwd	0.2478	0.2588	0.2972	0.86	1.00

*Table F6 — General Knowledge (HotpotQA): the eight grid cells with the lowest Completeness RMSE at 25 examples. This domain is ranked on Completeness rather than AUCROC because the gold adherence positive rate is 1.000 in every one of the 96 cells — there is no negative class, so AUCROC is undefined throughout. The last column reports judge coverage in its place. See §8.4.*